

04 — Manual GitHub sync guide (sanitized edition)

Updated: 2026-09-20

This edition omits internal project names, organization details, and dedicated environment names. The walkthrough and supplied screenshot retain this handbook's local folder path to make the steps concrete; use your own checkout path on another computer. The original internal Markdown and PDF guides remain local and are excluded from Git commits.

The example repository is HainanW/research-coding-handbook. GitHub stores files that have been committed and pushed; ignored local originals are not automatically backed up there.

1. Open the repository and check the remote

1.1. What does “open PowerShell in the project folder” mean?

It means **open a PowerShell command window and set its current working directory to your project folder**. Git uses that location to find the repository you want to work with. For this handbook, the project folder is research-coding-handbook, containing README.md, docs, and examples.

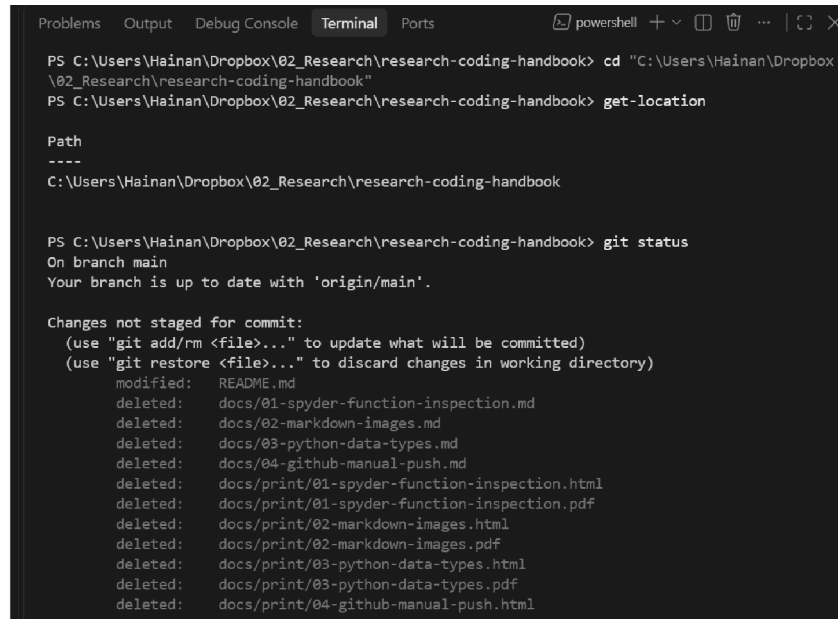
In VS Code, follow these steps:

1. Select **Terminal** → **New Terminal**. A panel for entering commands opens below the editor. If it uses another shell, select **PowerShell** from the dropdown beside the + button.
2. Enter the `cd` command below and press **Enter**. It uses the folder shown in the screenshot; replace the quoted path if your copy is elsewhere.
3. Enter `Get-Location` and press **Enter**. Check that its Path output is your research-coding-handbook folder.
4. Enter `git status` and press **Enter**. It reports the repository's changes; it does not commit or upload files.

Enter these commands one at a time in the terminal:

```
cd "C:\Users\Hainan\Dropbox\02_Research\research-coding-handbook"  
Get-Location  
git status
```

`cd` is an alias for `Set-Location`: it changes the current working directory without moving any files. Quotation marks keep a path containing spaces together. Enter commands in the **terminal panel**, not in a `.py` or `.md` file. The leading `PS ...>` is PowerShell's prompt; you do not type it yourself.



```

PS C:\Users\Hainan\Dropbox\02_Research\research-coding-handbook> cd "C:\Users\Hainan\Dropbox\02_Research\research-coding-handbook"
PS C:\Users\Hainan\Dropbox\02_Research\research-coding-handbook> get-location

Path
----
C:\Users\Hainan\Dropbox\02_Research\research-coding-handbook

PS C:\Users\Hainan\Dropbox\02_Research\research-coding-handbook> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        deleted:    docs/01-spyder-function-inspection.md
        deleted:    docs/02-markdown-images.md
        deleted:    docs/03-python-data-types.md
        deleted:    docs/04-github-manual-push.md
        deleted:    docs/print/01-spyder-function-inspection.html
        deleted:    docs/print/01-spyder-function-inspection.pdf
        deleted:    docs/print/02-markdown-images.html
        deleted:    docs/print/02-markdown-images.pdf
        deleted:    docs/print/03-python-data-types.html
        deleted:    docs/print/03-python-data-types.pdf
        deleted:    docs/print/04-github-manual-push.html

```

Figure 1. Change to the project folder with `cd`, confirm the location with `Get-Location`, and inspect changes with `git status`. This is the state when the screenshot was taken; your file list may differ. [View the full-size screenshot.](#)

Read the screenshot as follows:

- Path confirms the project directory, and On branch main identifies the current branch.
- Changes not staged for commit means there are changes that have not been staged. up to date with 'origin/main' compares commits with the locally recorded remote branch; it does not mean the working folder has no changes or that Git has just checked GitHub.
- After this handbook's filename changes, old tracked paths appear as deleted, while new paths may appear under Untracked files farther down the full output. Check that the new NN-main-... files exist and review the complete list; an old path marked deleted alone does not mean its content was lost.

Official references: [VS Code terminal basics](#) ^[1], [PowerShell current working directory](#) ^[2], and [Git status](#) ^[3].

1.2. Check the repository and remote

Once PowerShell is in the project folder, run these commands individually:

```

git status
git remote -v
git branch --show-current

```

For this repository, origin should point to <https://github.com/HainanW/research-coding-handbook.git>, and the branch is main.

1.3. First download on another computer

On another computer, download the repository from its intended parent directory:

```

git clone https://github.com/HainanW/research-coding-handbook.git
cd research-coding-handbook

```

The clone destination must not exist or must be empty. If a directory already contains files, compare, back up, and reconcile them first instead of overwriting them.

2. Get updates from GitHub

With a clean working tree, run:

```
git pull --ff-only origin main
```

If you have uncommitted changes, review and commit them using the next section first. If the branches have diverged, inspect `git log --oneline --graph --all` and reconcile the histories before pushing. Do not forcibly overwrite remote history.

2.1. What does `git pull --ff-only` mean?

`--ff-only` means **fast-forward only (allow only fast-forward updates)**.

```
git pull --ff-only
```

This means: fetch remote updates, and **update the local branch only when it can move directly forward along the current commit history**.

For example, the local branch is simply behind the remote branch:

```
Local:  A → B
Remote: A → B → C → D
```

It can update directly to D. This is called a “fast-forward.”

If the local and remote branches each have different new commits:

```
Local:  A → B → E
Remote: A → B → C → D
```

The histories have diverged. The command reports an error and stops integration, leaving you to decide how to handle it.

It suits the routine synchronization in the guide: **it avoids automatically creating a merge commit during a pull**. `ff` does not mean “force overwrite.”

Further Details

The letters in these diagrams represent example commits, not this repository's actual history. In the first example, moving the local branch from B to D uses the existing commits C and D; it does not create a new merge commit. In the second example, the local commit E is on a different path from C and D, so advancing along a single history is not possible.

`git pull` first fetches updates from the configured upstream branch, then tries to integrate them into the current branch. If `--ff-only` rejects the integration in the second example, the current local branch stays at E. The fetch step may already have downloaded remote commits and updated remote-tracking branches such as `origin/main`; the error does not mean that nothing was fetched.

The command at the start of section 2 includes `origin main`: `origin` selects the remote repository, and `main` selects its branch. The shorter `git pull --ff-only` uses the current local branch's configured upstream. If your current local branch is `main` and its upstream is `origin/main`, both commands fetch from the same branch and require fast-forward integration. Neither command switches your current branch.

Source: [Git documentation — `git pull`, including `--ff-only`](#)^[4].

3. Review, stage, and commit local files

```
git status --short
git diff
git add -- README.md
git diff --cached --stat
git diff --cached
git commit -m "Update handbook documentation"
```

Replace `README.md` with the files intended for this upload; multiple paths may be listed. Describe the actual change in the commit message. Open new files to review them, and inspect binary documents such as Word and PDF files rather than relying only on the diff summary.

Before publishing, check for passwords, tokens, internal addresses, or other material that should not be public. The two original `GitHub_Manual_Push_Guide` files at the repository root are excluded by `.gitignore`; do not force them into Git with `git add -f`.

If Git reports a missing author identity, configure your author name and verified email, or your GitHub-provided private email, for this repository. There is no need to change global settings for other projects.

4. Push to GitHub

```
git pull --ff-only origin main
git push origin main
```

Only continue after each command succeeds. Resolve push errors instead of using `--force`.

If you have a separately approved backup repository, configure and push to that remote separately. An editor's sync action does not necessarily update every remote. This example uses only `origin`.

5. Verify synchronization

```
git status
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

There should be no uncommitted changes, and the full local HEAD hash should match the remote `main` hash. A clean working tree alone does not prove your commits have been uploaded. Ignored files stay local and do not appear in ordinary `git status` output.

6. Directory ownership errors

If Git reports detected dubious ownership, first verify that the current directory is a local repository you trust, then run:

```
$TrustedRepo = (Get-Location).Path -replace '\\', '/'  
git config --global --add safe.directory $TrustedRepo
```

This adds an exception for the specific verified directory. Do not set `safe.directory` to `*`.

7. Troubleshooting

- non-fast-forward: the remote has commits you have not integrated. Fetch, inspect the differences, reconcile the histories, and retry.
- Conflicting files: retain both versions for comparison. Binary files often require a manual choice or separate copies.
- Missing uploads: check whether the files were staged, committed, and pushed, and whether `.gitignore` excludes them.
- Sensitive content committed by mistake: `.gitignore` does not remove existing history. Address exposed credentials first, then determine the appropriate history cleanup.

Daily sequence: pull updates → edit → review and commit → push → verify commit hashes.

References

1. <https://code.visualstudio.com/docs/terminal/basics>
2. <https://learn.microsoft.com/en-us/powershell/scripting/samples/managing-current-location>
3. <https://git-scm.com/docs/git-status>
4. <https://git-scm.com/docs/git-pull>